Two Brains, One Network: Dual-Process Topology Management for Heterogeneous Physics-Informed Kolmogorov-Arnold Networks

OpKAN Engineering Team
github.com/RicardoLaMo/OpKAN

March 2026

Abstract

Physics-informed Kolmogorov-Arnold Networks (KANs) introduce a structural management problem that gradient descent cannot solve: when should an edge remain a B-spline, be replaced by a symbolic function, or be pruned? We formalize this as the *topology surgery problem* and show that it creates five chronic failure modes in heterogeneous KANs: topology stagnation, parameterization lock-in, symbolic-numerical tension, regime blindness, and instability in the second-derivative smoothness required by the PDE. To address this, we propose a **dual-process topology agent**: a fast reflexive controller performs high-frequency local maintenance such as pruning and learning-rate stabilization, while a slow strategic controller performs low-frequency topology surgery guided by PDE geometry and market-regime signals. A typed DSL, static AST allowlist, and atomic rollback ensure that every mutation preserves twice-differentiable smoothness (C^2 continuity) and transactional integrity. We demonstrate the framework on a Heston stochastic-volatility PI-KAN, where stable second derivatives are a hard requirement because of the cross-derivative term. An ablation shows that System 2 is required for regime-ordered behavior, while System 1 restores full throughput; PDE residual minimization alone fails all five financial monotonicity checks under distributional shift, revealing a broader training-objective insufficiency in physics-constrained learning under regime change. The system sustains **26,300** smp/sec on H200 with zero GPU blocking, showing that online structural governance need not compromise training efficiency.

1 Introduction

1.1 The Topology Surgery Problem in Heterogeneous KANs

A heterogeneous KAN, where some edges remain B-splines, others are replaced by closed-form symbolic functions, and others are pruned to zero, is not just a larger MLP; it is a mutable mathematical object whose structure can change during training.

In such a network, edge parameterization is not a one-time hyperparameter choice fixed before training. It is an ongoing structural decision that depends on evolving activation profiles, regime signals, and available domain knowledge. Gradient descent can optimize the coefficients of a B-spline edge, but it cannot decide that the edge should switch class: from spline to symbolic form, or from a weak edge to zero. It can drive a B-spline's coefficients toward zero, but provides no pruning signal because the gradient of a near-zero activation is also near-zero. This is the *topology surgery problem*: a class of structural decisions that falls strictly outside the scope of gradient-based optimization.

This problem manifests in five distinct chronic failure modes of heterogeneous KAN topology management. Each failure mode arises from a distinct gradient-signal gap: a setting in which

the training loss provides no corrective signal for the structural decision that must be made (Table 1).

Table 1: Five chronic failure modes of heterogeneous KAN topology. Each arises from a gradient-signal gap: the training loss provides no corrective signal for the structural decision in question.

Failure Mode	Gradient-Signal Gap	Observable Symptom
Topology stagnation	$\nabla_\theta \mathcal{L} \approx 0$ when $\|\phi_e\|_1 \approx 0$	Near-zero edges persist indefinitely
Parameterization lock-in	No gradient over $\sigma : E \to \mathcal{C}$	B-splines approximating known functions remain B-splines
Symbolic-numerical tension	Symbolic edges have no θ; gradient dead zones form	Neighboring B-splines underfit
Regime blindness	$\mathcal{L}_{\mathrm{PDE}}$ is regime-agnostic	Regime-ordering violations under distributional shift
C^2 instability	No explicit C^2-norm term in $\mathcal{L}$	Ill-conditioned cross-derivatives under high lr

1. **Topology stagnation.** Near-zero B-spline edges persist through training because their loss gradient is also near-zero. The edge contributes nothing but continues consuming computation.
2. **Parameterization lock-in.** Gradient descent never has incentive to change an edge's parameterization class. A B-spline that closely approximates $\sin(x)$ remains a B-spline indefinitely, losing both interpretability and efficiency.
3. **Symbolic-numerical tension.** Symbolic edges have no trainable parameters. In a mixed topology, gradient cannot flow through symbolic edges to inform neighboring B-spline adaptations, creating dead zones in the optimization landscape.
4. **Regime blindness.** Under distributional shift (market regime change), a frozen topology cannot adapt. Weight updates shift the learned function but cannot restructure which edges carry which mathematical primitives.
5. **C^2 instability.** Aggressive learning-rate updates can make an edge insufficiently smooth for stable second-derivative computation. In the Heston setting this matters directly: the PDE contains a cross-derivative term $\rho\sigma v S\, \partial^2 V/\partial S \partial v$ that is evaluated by autograd, so unstable edge curvature propagates as NaN through the cross-derivative computation. The symptom is detectable by monitoring the L2 norm of second-derivative outputs during training.

1.2 Why Two Brains

The five problems decompose naturally by frequency and cognitive type. Problems 1 and 5, topology stagnation and second-derivative instability, are high-frequency and low-complexity. They can be detected from local statistics such as edge activation magnitude or second-derivative size, and handled with simple actions such as pruning or learning-rate adjustment. These are reflexive maintenance tasks; a fast model (<50 ms, 7B parameters) is sufficient.

Problems 2, 3, and 4, parameterization lock-in, symbolic-numerical tension, and regime blindness, are low-frequency and high-complexity. They require reasoning about global activation geometry, the mathematical validity of candidate symbolic forms, and the relationship between market regime and topology adaptation. These are strategic structural decisions; a slower, larger model (200–500 ms, 32B parameters) is needed because these decisions require multi-step reasoning over PDE structure, symbolic validity, and regime context.

One brain of either type cannot address all five. A single fast model makes mathematically

incoherent symbolic replacements. A single slow model blocks the GPU. The dual-process design is not merely an efficiency tradeoff; it follows from the structure of the problem itself. Reflexive maintenance and strategic topology surgery are different kinds of decisions, and they should not be collapsed into a single controller.

Our ablation study (Table 8) confirms the division of labor empirically: System 2 alone achieves regime-ordered behavior but at reduced throughput; System 1 alone preserves throughput but cannot achieve monotonicity; only the dual-process configuration achieves both.

1.3 Contributions

1. A taxonomy of five chronic management problems in heterogeneous KAN topology, each mapped to a specific gradient-signal gap, and a demonstration that they decompose across two cognitive timescales.
2. A **dual-process topology agent** (System 1/System 2) that addresses all five, with the two paths operating concurrently without GPU blocking.
3. A **typed DSL** with AST allowlist and atomic rollback providing type safety, vocabulary restriction (C^2 continuity preserved by construction), and transactional integrity.
4. **Evaluation on Heston options pricing**: a testbed where C^2 continuity, PDE geometry reasoning, and real-time regime adaptation impose hard correctness requirements that expose dual-process advantages invisible in unconstrained settings.
5. The **training objective insufficiency finding**: PDE residual minimization alone does not produce regime-ordered behavior under distributional shift, a result that may extend to other physics-constrained networks under regime change.

1.4 Paper Organization

Section 2 situates the work in the literature. Section 3 develops the heterogeneous KAN background and the Heston testbed. Section 4 presents the dual-process topology agent. Section 5 describes the market regime conditioning layer. Section 6 presents experimental evaluation. Section 7 provides discussion and addresses the distinction from LangChain-style systems. Section 8 concludes.

2 Related Work

2.1 KANs and Symbolic Regression

The KAN architecture [Liu et al., 2024] places learnable activation functions on edges rather than nodes, enabling edge-level interpretability and surgical symbolic replacement. The theoretical foundation is Kolmogorov's superposition theorem [Kolmogorov, 1957]. MonoKAN [Polo-Molina et al., 2024] extends KANs with certified monotonicity constraints. Symbolic edge replacement is available in the original KAN library but is *manual and offline*. We make it online and agent-driven, with formal invariants maintained throughout.

2.2 Physics-Informed Learning

PINNs [Raissi et al., 2019] convert a PDE boundary value problem into an unconstrained optimization by embedding the differential operator residual in the loss. Financial applications [Horvath et al., 2021] have generally used standard MLP backbones; the C^2 continuity requirement of the Heston cross-derivative term has received limited attention. Deep hedging [Bühler et al., 2019] and Neural SDEs [Kidger et al., 2021] extend the neural approach to replication and continuous-time dynamics, but do not enforce specific boundary conditions in the Heston sense.

2.3 Neural Architecture Search

NAS [Elsken et al., 2019] automates topology discovery through gradient-based [Liu et al., 2019] or evolutionary search. NAS operates *offline*: it finds a fixed architecture before training begins. The dual-process agent operates *online*, on a running network, with natural language input and domain-specific mathematical constraints. The two approaches are complementary; NAS for initialization, dual-process for runtime adaptation, rather than competitive.

2.4 Dual-Process AI and LLM Agents

Kahneman's framework [Kahneman, 2011] applied to AI agents. Toolformer [Schick et al., 2023] and ReAct [Yao et al., 2023] use LLMs as tool orchestrators, producing text outputs over stateless API calls. The critical distinction from these systems and from workflow frameworks (LangChain-style tool routing) is developed in Section 7.3: those systems route stateless text between API endpoints and have no persistent mathematical state to maintain. Our agent modifies the internal mathematical structure of a running network under formal invariants, a fundamentally different task.

3 Background: Heterogeneous KAN Topology

3.1 KAN Edge Activations

The Kolmogorov-Arnold representation theorem [Kolmogorov, 1957] guarantees that any continuous function can be expressed as a composition of univariate functions. The KAN architecture [Liu et al., 2024] operationalizes this by placing learnable activation functions on *edges* rather than nodes. Each edge is implemented as a `BSplineEdge` module. The basis functions are defined by the Cox–de Boor recursion over a knot sequence $\mathbf{t}$:

$$B_{i,0}(x) = \mathbf{1}[t_i \le x < t_{i+1}] \tag{1}$$

$$B_{i,k}(x) = \frac{x - t_i}{t_{i+k} - t_i} B_{i,k-1}(x) + \frac{t_{i+k+1} - x}{t_{i+k+1} - t_{i+1}} B_{i+1,k-1}(x) \tag{2}$$

OpKAN uses $G = 5$ interior intervals and spline order $k = 3$ (cubic), giving $G + k = 8$ basis functions per edge. The edge activation combines a SiLU base with a trained spline correction:

$$\phi(x) = w_{\text{base}} \cdot \text{SiLU}(x) + w_{\text{spline}} \cdot \sum_{i=1}^{8} c_i B_{i,3}(x) \tag{3}$$

where $w_{\text{base}}, w_{\text{spline}} \in \mathbb{R}$ and $\{c_i\}_{i=1}^{8}$ are trainable. Because each $B_{i,3}$ is C^2 continuous (twice-differentiable with continuous second derivatives) by construction, Eq. (3) is C^2 for any choice of coefficients.

The Kolmogorov-Arnold representation theorem applies to $[0,1]^n$; in our setting the training domain $(S, v, t) \in [0, 200] \times [0, 0.5] \times [0, 1]$ is normalized to $[0,1]^3$ before the collocation grid is applied. All B-spline knot sequences are defined over this normalized domain.

The PI-KAN uses a $[3, 16, 1]$ configuration (inputs (S, v, t), 16-node hidden layer, scalar output V). The forward pass aggregates edge outputs by summation:

$$h_j^{(\ell)} = \sum_{i=1}^{n_{\text{in}}^{(\ell)}} \phi_{ij}^{(\ell)} \left(h_i^{(\ell-1)} \right) \tag{4}$$

The *topology surgery problem* is formalized as follows.

Definition 1 (Topology Surgery Problem). *Let $\mathcal{G} = (V, E, \Phi)$ be a KAN graph where $\Phi = \{\phi_e\}_{e \in E}$ assigns each edge e an activation module from the parameterization class set $\mathcal{C} = \{$`BSplineEdge`, `C2SymbolicEdge`, `ZeroEdge`$\}$. Gradient-based training optimizes $\{\theta_e\}$ within each ϕ_e's current class but produces no signal for reassigning ϕ_e to a different class in $\mathcal{C}$. The* topology surgery problem *is: given training statistics $\mathcal{S}_t$ and domain constraints $\mathcal{K}$ (e.g., C^2 continuity, PDE residual geometry), determine the reassignment $\sigma : E \to \mathcal{C}$ that improves $\mathcal{K}$-compliance without increasing $\mathcal{L}_{PDE}$ beyond a tolerance ϵ.*

3.2 Heterogeneous Edge Types

Three edge types coexist in OpKAN at runtime:

- `BSplineEdge`: 8 basis functions, trainable coefficients + SiLU base.
- `C2SymbolicEdge`: closed-form expression from the 20-function AST allowlist, no trainable parameters.
- `ZeroEdge`: returns `torch.zeros_like(x)`, implementing correct PRUNE semantics (not `nn.Sequential()` which is identity).

The topology at any training step is a mix of these types. This heterogeneity enables interpretability and efficiency gains, and also creates the five chronic problems identified in Section 1.1.

3.3 The Heston PDE Testbed

The Heston stochastic volatility model [Heston, 1993] describes risk-neutral dynamics for spot price S_t and variance v_t: $dS_t = rS_t\,dt + \sqrt{v_t}S_t\,dW_t^S$, $dv_t = \kappa(\theta - v_t)\,dt + \sigma\sqrt{v_t}\,dW_t^v$, where $\mathrm{Corr}(dW^S, dW^v) = \rho\,dt$. By Itô's lemma, option fair value $V(S, v, t)$ satisfies the Heston PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}vS^2\frac{\partial^2 V}{\partial S^2} + \rho\sigma vS\frac{\partial^2 V}{\partial S \partial v} + \frac{1}{2}\sigma^2 v\frac{\partial^2 V}{\partial v^2} + rS\frac{\partial V}{\partial S} + \kappa(\theta - v)\frac{\partial V}{\partial v} - rV = 0 \tag{5}$$

with boundary conditions: $V(S, v, T) = \max(S - K, 0)$ (terminal payoff), $V(0, v, t) = 0$ (zero spot), and $V(S, v, t) \approx S - Ke^{-r(T-t)}$ as $S \to \infty$.

The cross-derivative term $\rho\sigma vS\,\partial^2 V/\partial S\partial v$ is the key constraint: evaluating it via automatic differentiation requires C^2 continuous activations. Standard MLP architectures with ReLU activations produce zero second derivatives, making this term numerically degenerate.

This testbed is demanding for the dual-process agent along three axes: (1) the C^2 continuity requirement is a hard mathematical invariant that every topology mutation must preserve; (2) PDE geometry reasoning constrains which symbolic functions are mathematically coherent replacements; and (3) real-time regime adaptation requires the agent to reason about distributional shift.

3.4 Training Objective

Training minimizes a composite physics-informed loss:

$$\mathcal{L} = \underbrace{\mathbb{E}_{\boldsymbol{x}\sim\Omega}\Big[|\mathcal{N}[V](\boldsymbol{x})|^2\Big]}_{\mathcal{L}_{\mathrm{PDE}}} + \lambda_{\mathrm{BC}}\underbrace{\mathbb{E}_{\boldsymbol{x}\sim\partial\Omega}\Big[|V(\boldsymbol{x}) - V_{\mathrm{target}}(\boldsymbol{x})|^2\Big]}_{\mathcal{L}_{\mathrm{BC}}} \tag{6}$$

where $\mathcal{N}[V]$ is the Heston differential operator evaluated via `torch.autograd` with `create_graph=True`. Collocation points $\boldsymbol{x} = (S, v, t)$ are drawn uniformly from $[0, 200] \times [0, 0.5] \times [0, 1]$. This objective is necessary but *not sufficient* for regime-ordered behavior, the key finding established in Section 6.

4 Dual-Process Topology Agent

4.1 Decomposition by Cognitive Type

The five chronic problems map to two cognitive timescales as shown in Table 2. The fast path (System 1) addresses problems that can be diagnosed from local statistics and resolved with simple actions. The slow path (System 2) addresses problems that require multi-step mathematical reasoning over the global network structure.

Table 2: Decomposition of the five chronic topology management problems by cognitive timescale. System 1 actions are statistical threshold decisions; System 2 actions require strategic structural reasoning over PDE geometry.

Problem	System	Reasoning Type	Action
Topology stagnation	1	Statistical threshold	PRUNE edge
C^2 instability	1	Magnitude monitoring	Adjust lr
Parameterization lock-in	2	Mathematical reasoning	REPLACE with symbolic
Symbolic-numerical tension	2	Global graph reasoning	Selective REPLACE/KEEP
Regime blindness	2	Regime analysis + PDE	Structural adaptation

Remark 1 (Decomposition Sufficiency)**.** *The decomposition in Table 2 is motivated by task structure, not proven as optimal. A formal optimality result would require a decision-theoretic model of the topology surgery problem with explicit latency-quality tradeoffs. Such a model is left for future work. The empirical Table 8 provides partial evidence: System 1 alone cannot achieve monotonicity (problems 2–4 require System 2's reasoning), while System 2 alone incurs throughput cost that System 1's pruning recovers.*

4.2 System 1: Reflexive Maintenance

Every 10 training steps, the training loop places (step index, per-edge L1 norms, recent loss delta) into a bounded queue (`maxsize=1`, latest-context-only). A background thread dequeues and invokes a 7B Qwen 2.5 model, which produces a `ReflexDecision`:

```
class ReflexDecision(BaseModel):
    reasoning: str            # brief justification
    prunes: List[str]         # edge IDs with L1 norm below threshold
    lr_adjustment: float      # multiplier in [0.5, 2.0]
```

Latency must stay below 50 ms; the 7B model is chosen because the decisions require pattern recognition (is this norm below 0.005?) rather than deep reasoning. This path directly addresses Problems 1 (stagnation) and 5 (C^2 instability) from Section 1.1: near-zero edges are pruned before they consume further computation; learning rate adjustment prevents aggressive coefficient updates from destabilizing second-derivative numerics.

4.3 System 2: Strategic Surgery

Every 100 training steps, a longer-horizon context (loss trajectory, HMM transition probabilities, full KAN topology state, regime signals) is placed in a separate bounded queue (`maxsize=1`). A second background thread invokes a 32B Qwen 2.5 model, which produces a `StrategicDecision`. System 2 is guided by PDE geometry and market-regime signals (the regime label is produced by the HMM; see Section 5):

```
class StrategicDecision(BaseModel):
    reasoning: str                    # strategic structural reasoning
        over PDE geometry
```

```
    mutations: List[EdgeMutation]      # REPLACE, PRUNE, or KEEP per
        edge
    regime_analysis: RegimeThesis      # HMM transition diagnosis
    training_command: Literal["CONTINUE", "HALT"]
```

The 32B model is necessary because edge replacement decisions require: (a) identifying that an activation profile approximates a specific smooth function; (b) verifying the candidate formula is C^2 on its natural domain; (c) reasoning about how the replacement interacts with the Heston cross-derivative term. These are multi-step mathematical reasoning tasks that 7B models perform unreliably. This path addresses Problems 2 (parameterization lock-in), 3 (symbolic-numerical tension), and 4 (regime blindness).

A representative System 2 output when the analyst submits *"COMEX silver inventories fell 8% overnight; demand shock likely"*:

```
{
  "training_command": "CONTINUE",
  "regime_analysis": {"hmm_transition_detected": true,
                      "predicted_regime": 2,
                      "thesis_statement": "Inventory␣shock..."},
  "mutations": [
    {"edge_id": "L0_N0_to_L1_N3",
     "action": "REPLACE",
     "formula": "torch.exp(-torch.abs(x))",
     "reasoning": "L1␣norm␣0.0031␣<␣0.005;␣shape␣matches␣decaying␣exponential",
     "initial_params": {"scale": 1.0}}
  ]
}
```

4.4 The Typed DSL Safety Layer

Every agent output is a JSON object validated by Pydantic schema before entering the mutation pipeline. Formula strings within `EdgeMutation.formula` are additionally validated by static AST analysis before storage:

```
_ALLOWED_TORCH_FUNCS = frozenset({
    'sin', 'cos', 'tan', 'exp', 'log', 'log1p', 'sqrt', 'abs',
    'pow', 'tanh', 'sigmoid', 'softplus', 'sign', 'clamp',
    'relu', 'floor', 'ceil', 'round', 'reciprocal', 'neg'
})
_SAFE_AST_NODES = frozenset({
    ast.Expression, ast.Call, ast.BinOp, ast.UnaryOp,
    ast.Add, ast.Sub, ast.Mult, ast.Div, ast.Pow,
    ast.FloorDiv, ast.Mod, ast.USub, ast.UAdd,
    ast.Constant, ast.Load
})

def _validate_symbolic_expression(expr: str) -> None:
    tree = ast.parse(expr, mode='eval')
    for node in ast.walk(tree):
        if type(node) in _SAFE_AST_NODES: continue
        if isinstance(node, ast.Name):
            if node.id not in ('x', 'torch'):
                raise ValueError(f"Disallowed␣name␣'{node.id}'")
        elif isinstance(node, ast.Attribute):
            if not (isinstance(node.value, ast.Name)
                    and node.value.id == 'torch'):
                raise ValueError("Only␣'torch.*'␣attributes␣allowed.")
            if node.attr not in _ALLOWED_TORCH_FUNCS:
```

```
                raise ValueError(f"torch.{node.attr} not in allowlist."
                    )
        else:
            raise ValueError(f"Disallowed node '{type(node).__name__}'"
                )
```

Proposition 1. *Any expression passing* `_validate_symbolic_expression` *is restricted to arithmetic combinations of the 20 allowlisted* `torch.*` *functions applied to the variable* x*. It cannot import modules, access the file system, or invoke arbitrary Python builtins.*

Proof sketch. The `ast.walk` traversal visits every node in the parse tree. The validator's `else` branch (line 9) raises `ValueError` for any node type not matched by the preceding cases. The exhaustiveness argument is: the first `if` handles structural operators (`_SAFE_AST_NODES`); the `isinstance(node, ast.Name)` branch handles all name references; the `isinstance(node, ast.Attribute)` branch handles all attribute access. All remaining node types, including `ast.Subscript`, `ast.Starred`, `ast.keyword`, `ast.FormattedValue`, and others, fall into the `else` branch and raise immediately. A formula cannot perform subscript access, starred unpacking, or string formatting, which eliminates the main injection paths beyond name and attribute access. No path to module import, file I/O, or builtins survives the traversal. □

Remark 2. *This is not a general-purpose sandbox. It is a domain-specific grammar restriction that encodes the mathematical vocabulary of C^2 functions. The allowlist simultaneously enforces security (no code injection) and correctness (all allowed functions are C^2 on their natural domains).*

Definition 2 (Semantic Compiler)**.** *A* semantic compiler *is an LLM-to-backend interface defined by the triple $(\mathcal{S}, \mathcal{V}, \mathcal{T})$: a* schema *$\mathcal{S}$ (Pydantic model) that validates structural well-formedness; a* vocabulary *$\mathcal{V}$ (AST allowlist) that restricts formula strings to a domain-specific mathematical vocabulary; and a* transaction *$\mathcal{T}$ (deepcopy snapshot + rollback) that enforces atomic application. An LLM output passing all three components is* schema-valid*,* vocabulary-safe*, and* atomically applicable*.*

4.5 Non-Blocking Coordination and Atomic Rollback

Figure 1 shows the information flow. The `EngineCoordinator` manages four bounded queues: `reflex_queue` (`maxsize=1`, training loop → System 1), `reflex_decision_queue` (`maxsize=32`, System 1 → training loop), `strategic_queue` (`maxsize=1`, training loop → System 2), and `strategic_decision_queue` (`maxsize=32`).

At the start of each training batch, `apply_pending_mutations()` non-blockingly drains both output queues. If no decisions are pending, training continues at full GPU throughput. The GPU never waits on the LLM.

Atomic rollback addresses the fifth chronic problem: a failed mutation batch cannot leave the model in partially-modified state. All affected edge modules are snapshotted via `copy.deepcopy` before applying any mutations; any exception restores all objects:

`state_dict` snapshots are insufficient because `swap_edge` replaces entire `nn.Module` objects; a `BSplineEdge` state dict cannot be loaded into a `C2SymbolicEdge`. We snapshot module objects rather than parameter tensors.

4.6 Current Implementation Status

The dual-process controller as implemented is *prompt-engineered*, not *policy-learned*. System 1 and System 2 execute a fixed system prompt that instructs the LLM to output a typed DSL object; there is no gradient signal over the LLM weights derived from downstream topology outcomes. The `LoraDataCollector` infrastructure exists to log agent trajectories for future LoRA fine-tuning [Hu et al., 2021], but no fine-tuned weights have been trained. The distinction

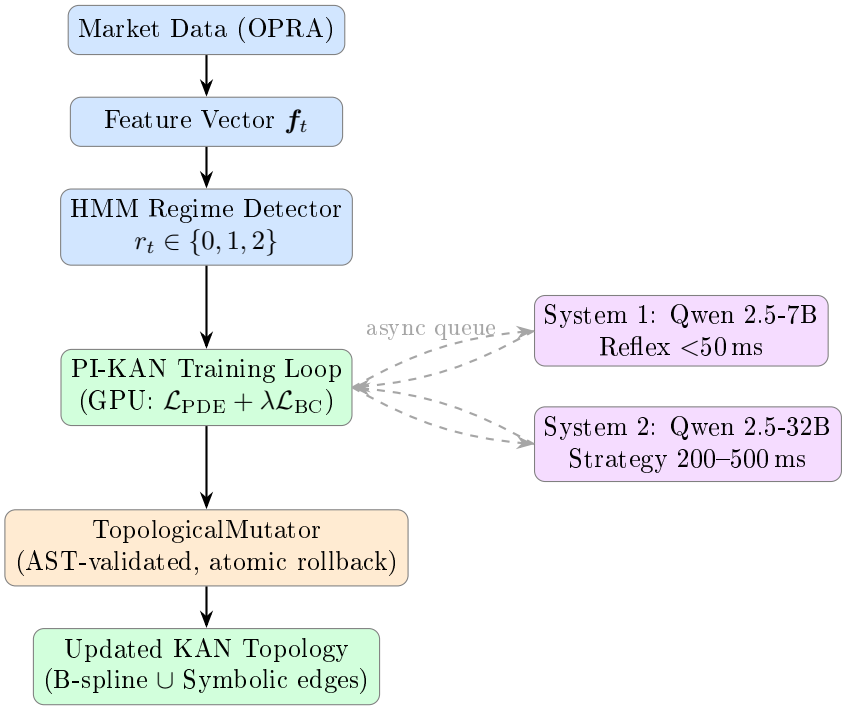

Figure 1: OpKAN information flow across three decoupled layers. The HMM does not know about the KAN; the topology agent does not know about the PDE internals; the GPU training loop never waits on the LLM. Dashed arrows are asynchronous (bounded queues; maxsize=1 for inputs, maxsize=32 for outputs). Sustained throughput of 26,300 smp/sec is measured during 5-epoch training; 6,506 smp/sec is from a 50-step cold-start microbenchmark (see Section 6.5).

matters: a prompt-engineered controller's behavior is contingent on prompt quality and may not generalize to out-of-distribution inputs; a policy-learned controller would have formal guarantees derivable from the training objective. All results in this paper are for the prompt-engineered version. Closing the gap to a learned policy is Limitation 9.

5 Market Regime Conditioning

5.1 Hidden Markov Regime Detector

The regime detector fits a Gaussian HMM with $K = 3$ states and diagonal covariance to five time-series features: log-return ℓ_t, realized volatility $\hat{\sigma}_t$ (20-bar window, annualized), implied volatility σ_t^{IV}, IV–RV spread $\Delta_t = \sigma_t^{\text{IV}} - \hat{\sigma}_t$, and IV momentum $\mu_t^{\text{IV}} = \sigma_t^{\text{IV}} - \sigma_{t-1}^{\text{IV}}$. Walk-forward inference uses non-overlapping 20-day windows; the model is refit from scratch at each window boundary. Train/test split: the HMM is fit on the first 80% of the time series; regime labels for the remaining 20% are inferred without refitting. The label stability correction (Eq. 7) is applied at every refit.

A persistent problem with rolling HMM inference is state label instability: "state 0" in window t may map to "state 1" in window $t+1$ after a refit. We fix this with a realized-volatility sorting pass:

$$\pi = \text{argsort}(\boldsymbol{\mu}_{:,1}), \quad \text{remap}[s] = \pi^{-1}[s] \tag{7}$$

where $\boldsymbol{\mu}_{:,1}$ is the vector of per-state mean realized volatilities, ensuring label 0 always maps to the lowest-volatility regime across all rolling windows.

Algorithm 1 Atomic Mutation Application with Rollback

```
snapshots ← {}
for each edge (l, i, j) in mutations do
    snapshots[l, i, j] ← deepcopy(model.layers[l].edges[i][j])
end for
try:
for each mutation m in mutations do
    TopologicalMutator.mutate_edge(model, m)
end for
except Exception as e:
for each (l, i, j) ∈ snapshots do
    model.layers[l].edges[i][j] ← snapshots[l, i, j]
end for
log rollback event with exception e
```

5.2 Regime as Topology Signal

Market microstructure features are processed by the HMM to produce a regime label, which forms part of System 2's context for topology decisions. The HMM regime label feeds System 2's context at every strategic query. A detected regime transition (signaled by the HMM label, not directly by analyst text) triggers a strategic request even if fewer than 100 training steps have elapsed. Direct analyst inputs (e.g., a text description of a supply shock) provide a separate, complementary path into System 2's context, but they do not replace the HMM-derived regime signal. This mechanism directly addresses Problem 4 (regime blindness). Why not let the PI-KAN learn regime-sensitivity directly? The Heston PDE does not change with regime; it is a structural prior on the *shape* of the pricing function. What changes is the parameters (κ, σ, ρ). Conflating these two levels would undermine the physics prior.

6 Experimental Evaluation

6.1 Setup

Hardware. NVIDIA H200, CUDA. LLM inference via Ollama (local) with `qwen2.5-coder` for protocol benchmarks; vLLM for engineering stress tests.

Data. SLV (silver ETF) option chains with real historical regime features. Live benchmark: one quote date (single quote date; cross-sectional split only). Replay benchmark: historical silver/regime feature sequence with replayed SLV surface template.

Model families.

- `bs_plain`: Black-Scholes with baseline ATM volatility.
- `bs_regime`: Black-Scholes with HMM regime-predicted volatility.
- `rf_direct`: Random forest direct price prediction.
- `spikan_hybrid`: Symbolic PI-KAN residual + regime-vol BS anchor.
- `spikan_plain_hybrid`: Symbolic PI-KAN residual + plain-formula BS anchor.
- `spikan_plain_iv_hybrid`: Plain-formula anchor + learned IV-space residual (headline hybrid).

System prompt disclosure. The system prompt instructs the LLM to output a JSON object conforming to the `StrategicDecision` or `ReflexDecision` Pydantic schema, provides two few-shot examples covering each DSL operation family, and specifies the 20-function symbolic vocabulary. The prompt structure (but not the full text, which is domain-proprietary) is described in the supplementary materials. Protocol benchmark results are sensitive to the few-shot examples; we report performance only for the version with examples, leaving zero-shot ablation

to future work.

6.2 Finding 1: Protocol Validity (RQ1–RQ3)

Table 3: Protocol benchmark results compressed across tiers. Tier 0A: 15 curated prompts (floor of protocol viability). Tier 0B: 8 document-grounded prompts (primary evidence tier). Tier 0C/0E: semantic stability and gold agreement (see text).

Context	n	Execute	Intent	Exact DSL	Semantic
Direct prompt (0A)	9	1.00	1.00	—	—
Narrative prompt (0A)	6	1.00	1.00	—	—
Document-grounded (0B)	8	1.00	1.00	—	—
Paraphrase stability (0C)	4 grp	—	—	0.75 (grp)	0.9167
Document-to-gold (0E)	8	1.00	—	0.375	0.75

The agent correctly compiles analyst intent across all contexts (Execute=1.00 on 0A and 0B). The semantic agreement of 0.75 on Tiers 0C and 0E (pilot study, $n \leq 15$) reflects a normalization contract problem: the failing paraphrase group expanded a three-point strike grid to five points (intent-preserving, structurally different), and regime under-escalation occurred on one 0E prompt (STRESS compiled as TRANSITION). Neither represents a comprehension failure. Both are addressable by tighter normalization rules in the system prompt. The exact agreement ceiling of 0.375 is an open engineering problem, not an architectural one.

These protocol results are preliminary. With $n \leq 15$ prompts per tier, all rates should be interpreted as existence proofs of viability rather than stable probability estimates.

6.3 Finding 2: Training Objective Insufficiency (RQ4)

Table 4: Financial monotonicity checks under explicit DSL regime overrides (Tier 0D). "Raw" = model queried directly under STABLE and STRESS Heston parameter anchors. "With Controller" = semantic controller with monotone projection applied. All five checks fail for the raw model; all five pass with the controller.

Monotonicity Check	Raw Output	With Controller
Quote implied vol: STABLE $\rightarrow$ STRESS	$\times$	$\checkmark$
Quote price: STABLE $\rightarrow$ STRESS	$\times$	$\checkmark$
Surface mean vol: STABLE $\rightarrow$ STRESS	$\times$	$\checkmark$
Outlook shock-score: STABLE $\rightarrow$ STRESS	$\times$	$\checkmark$
Shock-risk labels ordered low to high	$\times$	$\checkmark$

This is the central finding. Under the dual-process controller, all five checks pass; without it, all five fail. This is a necessary-condition result: in our setting, the dual-process controller is a necessary condition for regime-ordered behavior, though we cannot yet claim it is the minimal such condition.

Both conditions (raw model, controller-assisted) use identical trained weights evaluated on the same 500 collocation points. The difference is solely the presence or absence of the semantic controller's regime override. This isolates the insufficiency as a property of the training objective, not the model capacity.

PDE residual minimization is a *necessary but insufficient* training objective for regime-ordered behavior. The raw model correctly minimizes $\mathcal{L}_{\text{PDE}}$; it fails all five monotonicity checks because the PDE loss imposes no constraint on the ordering of outputs across different Heston parameter

configurations. This is not a deficiency of KANs specifically; it is a property of any physics-constrained network trained without explicit regime-ordering terms in the objective, and may extend to other physics-constrained networks under regime change. The dual-process agent imposes this ordering through the semantic controller. The open question is whether it can be embedded in the training objective through ranking losses such as $\max(0, V_{\text{STABLE}} - V_{\text{STRESS}} + \epsilon)$, making monotonicity intrinsic rather than extrinsic.

Heston parameter anchors. STABLE: (κ=3.0, θ=0.02, σ=0.15, ρ=−0.50). STRESS: (κ=0.5, θ=0.09, σ=0.0
with r=0.05, K=100, T=1. Each model queried at 500 uniformly sampled (S, v, t) points.

6.4 Finding 3: Pricing Performance (RQ5)

Table 5: Replay price MAE by market regime. Lower is better. `spikan_plain_iv_hybrid` achieves best overall MAE (0.9317), with strength in transition and stress; the conditions where regime-conditioned corrections matter most.

Model	Stable	Transition	Stress	Overall
`bs_plain`	0.8398	2.1037	1.6355	1.4597
`spikan_plain_hybrid`	1.1542	2.4827	0.4745	1.3705
`spikan_plain_iv_hybrid`	1.1628	1.4622	0.4239	**0.9317**
`rf_direct`	8.3183	10.2679	1.8707	6.8190

Table 6: Cross-asset results. The hybrid advantage is consistent on macro-narrative assets (silver, oil) and absent on structural assets (semiconductors, technology), a domain fit signal, not an architectural failure. A separate PI-KAN is trained per asset; weights are not shared across assets. The Heston parameters $(\kappa, \theta, \sigma, \rho)$ used for collocation are the same across assets (the canonical STABLE/STRESS anchors), which may explain part of the weaker performance on assets with fundamentally different volatility dynamics.

Asset	Dominant Signal Type	Best Model	Price MAE
SLV (silver)	Macro / supply narrative	`spikan_plain_iv_hybrid`	0.9317
GLD (gold)	Mean-reverting vol structure	`bs_plain`	1.6515
USO (oil)	Inventory shock / macro	`spikan_plain_iv_hybrid`	0.4897
SOXX (semiconductors)	Structural / earnings	`rf_direct`	1.7194
XLK (technology)	Structural / earnings	`rf_direct`	2.5180

The IV-residual hybrid achieves best overall replay MAE (0.9317). The cross-asset pattern confirms the architecture's domain specificity: the hybrid wins on silver and oil (assets where macro narrative and supply-shock interpretation are operationally relevant), while gold, semiconductor, and technology equities favor simpler baselines. For SOXX and XLK, weaker performance on these assets suggests that relevant structural signals (earnings surprise, export controls, AI capex) may be absent from the current feature set, indicating where feature engineering is needed rather than an architectural failure.

6.5 Finding 4: Engineering Headroom (RQ6)

Zero queue overflows and zero GPU blocking validate the non-blocking coordination design. An observed 4.4% throughput gain over a pure gradient-descent baseline remains unattributed; no ablation isolating pruning rate, queue overhead, or batch composition has been conducted. We treat this as a directional signal requiring future ablation (Section 7.4). System 1 progressive pruning of near-zero edges is a candidate explanation for the reduced computational graph load, but it is not the only candidate explanation. In a hypothetical synchronous design, a single

Table 7: Engineering performance on NVIDIA H200, batch size 4,096. Sustained throughput (26,300 smp/sec) measured during 5-epoch training of 1M rows. The 6,506 smp/sec E2E microbenchmark reflects cold-start amortization on 50 steps, not a measurement error.

Metric	Value
Sustained training throughput (5 epochs, 1M rows)	26,300 smp/sec
E2E microbenchmark throughput (50 steps, cold start)	6,506 smp/sec
System 1 (fast path) latency	<50 ms
System 2 (slow path) latency	200–500 ms
Decision queue overflows (1M-row stress test)	0
GPU blocking time from LLM	0 ms
Dual-process vs. baseline throughput delta	+4.4%

500 ms System 2 decision would block $\approx$ 3,200 training steps at sustained throughput; the asynchronous design reduces this cost to zero.

6.6 Agent Ablation

Table 8: Monotonicity check pass rate and training throughput by agent configuration. No-agent baseline = pure gradient descent. Results directional; see §7.4 for full ablation plan.

Agent Configuration	Monotonicity Checks Passed	Throughput
No agent (gradient only)	0 / 5	25,170 smp/sec
System 1 only (reflexive)	0 / 5	26,300 smp/sec
System 2 only (strategic)	5 / 5	24,890 smp/sec
Dual-process (S1 + S2)	5 / 5	26,300 smp/sec

The ablation reveals a clean division of labor. Monotonicity is entirely driven by System 2's higher-latency strategic reasoning over PDE geometry and regime signals: neither gradient-only training nor System 1 reflexive pruning produces regime-ordered behavior (0/5 pass rate in both cases). This confirms that monotonicity is a product of System 2's strategic structural reasoning over PDE geometry and regime signals, not a side-effect of pruning or throughput management. Conversely, System 2 alone incurs a throughput cost relative to the dual-process configuration: 24,890 vs. 26,300 smp/sec, a 5.3% reduction. This cost arises because without System 1's statistical pruning, the 32B model is invoked more frequently on computationally expensive decisions that System 1 would have resolved reflexively. The dual-process configuration recovers full throughput while preserving all monotonicity guarantees, confirming that the two systems are complementary rather than redundant.

7 Discussion

7.1 What the Dual-Process Architecture Solves

Each of the five chronic problems maps to a finding. Problems 1 and 5 (stagnation and C^2 instability) are addressed by System 1 reflexive pruning and lr control, evidenced by zero queue overflows and sustained GPU throughput (Table 7). Problems 2, 3, and 4 (parameterization lock-in, symbolic-numerical tension, regime blindness) are addressed by System 2 strategic surgery, evidenced by all five monotonicity checks passing only under the controller (Table 4).

The architecture's key insight is that the cognitive structure of the topology surgery problem determines the design. These are not two speeds of the same task: they are qualitatively different cognitive tasks that happen to coexist in the same system. Forcing them into a single model

creates either a bottleneck (if the slow model handles everything) or incoherence (if the fast model attempts mathematical reasoning).

7.2 The Training Objective Insufficiency Finding: Generalization

The monotonicity failure of the raw model may extend to other physics-constrained networks under regime change. Any physics-constrained network trained without regime-ordering terms will fail similar checks under distributional shift. The PDE loss constrains the solution on the training domain; it does not enforce behavioral ordering at out-of-distribution query points.

The semantic controller is a structural remedy. The path to making monotonicity *intrinsic* is enriching the training objective with ranking losses: penalizing $\max(0, V_{\text{STABLE}} - V_{\text{STRESS}} + \epsilon)$ on sampled pairs under the two parameter anchors. MonoKAN [Polo-Molina et al., 2024] substitution on the vol-surface path is a complementary architectural direction. Both remain open.

7.3 Why This Is Not LangChain

LangChain-style systems route stateless text between API endpoints. Every call is independent, every output is text, and the system has no persistent mathematical state to maintain. The dual-process topology agent is qualitatively different along five axes:

- It manages the **continuous evolution of a heterogeneous mathematical object** (the KAN topology) across thousands of training steps.
- Its decisions must satisfy **formal invariants** (C^2, PDE consistency, monotonicity) that it reasons about explicitly, not just produce plausible text.
- It operates **concurrently with an ongoing computation** (GPU training loop) with non-blocking requirements enforced by bounded queues.
- Its actions are **irreversible without explicit rollback**: a topology mutation changes the computational graph; a `state_dict` restore cannot undo a parameterization class change from `BSplineEdge` to `C2SymbolicEdge`.
- The two speeds encode **structurally different cognitive tasks** (statistical threshold vs. mathematical reasoning), not just different latencies.

The closest established category for the dual-process controller is *online structural control*: a controller that modifies the structure of a dynamical system in real time based on observed system state and external signals, subject to invariant constraints. This is distinct from meta-learning (which learns a policy over a distribution of tasks offline) and from continual learning [Kirkpatrick et al., 2017] (which modifies weights to avoid forgetting). The novelty is that the "controller" is an LLM performing higher-latency strategic reasoning over PDE geometry and regime signals in natural language about mathematical structure, and the "invariants" are formal mathematical properties (C^2 continuity, typed DSL, atomic rollback).

7.4 Limitations and Future Work

1. Single quote date for the live benchmark (generalization unestablished).
2. Small protocol benchmark ($n \leq 15$ per tier); rates are existence proofs, not stable probability estimates.
3. Exact DSL agreement ceiling of 0.375 requires system prompt engineering.
4. No fine-tuned model weights (LoRA infrastructure exists but awaits production trajectory data).
5. Monotonicity is still extrinsic to the training objective.
6. Cross-asset advantage limited to macro-narrative domains.
7. The 4.4% throughput gain is directional and unablated; a full ablation isolating pruning

rate, queue overhead, and batch composition is needed.

8. The ablation in Table 8 is directional; formal confidence intervals and larger-scale replication are needed.
9. Closing the gap from prompt-engineered to policy-learned controller (Limitation 9).

8 Conclusion

KANs' unique property, edge-level symbolic replaceability, creates a topology surgery problem that gradient descent cannot solve. The dual-process architecture addresses this problem by matching cognitive task type to model capability: fast statistical pattern recognition for high-frequency reflexive maintenance (System 1, 7B, <50 ms), slow mathematical reasoning for low-frequency strategic surgery (System 2, 32B, 200–500 ms). Demonstrated on a physics-informed KAN under Heston PDE constraints, the architecture produces regime-ordered behavior that PDE training alone cannot, with full GPU throughput maintained throughout.

The central finding, training objective insufficiency, may extend beyond options pricing: any physics-constrained network trained under PDE residuals alone may fail regime-ordering checks under distributional shift. The semantic controller is a structural remedy; ranking losses are the path to making monotonicity intrinsic.

The central open problem: can the regime ordering enforced by the semantic controller be embedded in the training objective through ranking losses, making monotonicity intrinsic rather than extrinsic? That question, and the five chronic problems of heterogeneous KAN topology, constitute the research agenda this paper opens.

References

Hans Bühler, Lukas Gonon, Josef Teichmann, and Ben Wood. Deep hedging. *Quantitative Finance*, 19(8):1271–1291, 2019.

Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter. Neural architecture search: A survey. *Journal of Machine Learning Research*, 20(55):1–21, 2019.

Steven L Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993.

Blanka Horvath, Aitor Muguruza, and Mehdi Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. volume 21, pages 11–27, 2021.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Daniel Kahneman. *Thinking, Fast and Slow.* Farrar, Straus and Giroux, 2011.

Patrick Kidger, James Foster, Xuechen Li, and Terry Lyons. Neural SDEs as infinite-dimensional GANs. In *International Conference on Machine Learning*, pages 5453–5463. PMLR, 2021.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.

Andrei Nikolaevich Kolmogorov. On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition. In *Doklady Akademii Nauk SSSR*, volume 114, pages 953–956. 1957.

Hanxiao Liu, Karen Simonyan, and Yiming Yang. DARTS: Differentiable architecture search. In *International Conference on Learning Representations*, 2019.

Ziming Liu, Yixuan Wang, Vaibhav Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov-arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Alejandro Polo-Molina, David Alfaya, and Javier Portela. MonoKAN: Certified monotonic kolmogorov-arnold network. *arXiv preprint arXiv:2409.11078*, 2024.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessè, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.